Expo Router আয়ত্ত করুন

React Native অ্যাপের আধুনিক নেভিগেশন

Part 1 — অধ্যায় ১ থেকে ৫

BSc শিক্ষার্থী ও জুনিয়র ডেভেলপারদের জন্য সম্পূর্ণ গাইড

ভূমিকা (Preface)

মোবাইল অ্যাপ ডেভেলপমেন্টের দুনিয়ায় React Native একটি বিশেষ স্থান অধিকার করেছে। এটি দিয়ে একবার কোড লিখে Android এবং iOS উভয় প্ল্যাটফর্মের জন্য অ্যাপ তৈরি করা সম্ভব। কিন্তু React Native দিয়ে কাজ করার সময় নতুন ডেভেলপারদের সবচেয়ে বেশি সমস্যা হয় Navigation নিয়ে — অর্থাৎ, এক স্ক্রিন থেকে অন্য স্ক্রিনে কিভাবে যাওয়া যায়, কিভাবে URL-এর মতো করে রুট ম্যানেজ করা যায়।

এই বইটি লেখা হয়েছে তাদের জন্য যারা Expo Router শিখতে চান — যারা হয়তো ওয়েব ডেভেলপমেন্টে Next.js ব্যবহার করেছেন, অথবা React Native একদম নতুন শুরু করছেন। Expo Router হলো একটি আধুনিক, file-based routing সিস্টেম যা Next.js-এর মতো কাজ করে, কিন্তু মোবাইল অ্যাপের জন্য।

এই বইয়ে আপনি শিখবেন:

- Expo এবং Expo Router কী এবং কেন এটি আধুনিক মোবাইল ডেভেলপমেন্টের পছন্দের tool
- File-based routing কিভাবে কাজ করে এবং কিভাবে এটি পুরনো পদ্ধতির চেয়ে ভালো
- Stack, Tab, এবং Drawer navigation কিভাবে implement করতে হয়
- Authentication, Protected Routes, API Integration সহ real-world প্রজেক্ট
- Performance optimization, Testing, এবং Deployment

প্রতিটি অধ্যায়ে real-world উদাহরণ, সম্পূর্ণ কোড, এবং ব্যাখ্যা দেওয়া হয়েছে। অধ্যায় শেষে Key Takeaways, Common Mistakes, Best Practices এবং Exercise দেওয়া আছে যাতে আপনি নিজে চর্চা করতে পারেন।

এই বইটি পড়তে আপনার JavaScript এবং React-এর প্রাথমিক জ্ঞান থাকলেই যথেষ্ট। React Native বা Expo সম্পর্কে আগের অভিজ্ঞতা না থাকলেও চলবে — আমরা একদম শুরু থেকে শুরু করব।

শুভকামনা রইল আপনার React Native যাত্রায়! 🚀

সূচিপত্র (Table of Contents)

Part 1: মৌলিক ধারণা

অধ্যায় ১ — Expo এবং React Native পরিচিতি

অধ্যায় ২ — Expo Router কী এবং কেন ব্যবহার করবেন

অধ্যায় ৩ — Installation এবং Setup

Part 2: Routing এর মূল ধারণা

অধ্যায় ৪ — File-based Routing ব্যাখ্যা

অধ্যায় ৫ — Navigation Basics (Link, useRouter)

অধ্যায় ৬ — Dynamic Routing ([id].js)

অধ্যায় ৭ — Layouts এবং Nested Routing

Part 3: Navigation System

অধ্যায় ৮ — Stack Navigation

অধ্যায় ৯ — Tab Navigation

অধ্যায় ১০ — Drawer Navigation (Advanced)

Part 4: বাস্তব ব্যবহার

অধ্যায় ১১ — Authentication System

অধ্যায় ১২ — Protected Routes

অধ্যায় ১৩ — Error Handling

Part 5: Data Handling

অধ্যায় ১৪ — API Integration

অধ্যায় ১৫ — State Management

অধ্যায় ১৬ — Form Handling

Part 6–9: UI/UX, Advanced, Testing, Project

অধ্যায় ১৭-২৭ — (পরবর্তী ভলিউমে)

অধ্যায় ১: Expo এবং React Native পরিচিতি

এই অধ্যায়ে আমরা জানব React Native কী, কেন এটি এত জনপ্রিয়, Expo কীভাবে React Native-কে আরো সহজ করে তোলে, এবং কিভাবে একটি simple Expo অ্যাপ তৈরি করা যায়।

১.১ React Native কী?

ওয়েব ডেভেলপমেন্টে আমরা HTML, CSS, এবং JavaScript ব্যবহার করি। কিন্তু মোবাইল অ্যাপ তৈরিতে Android-এর জন্য Kotlin/Java এবং iOS-এর জন্য Swift/Objective-C দরকার হয়। এর মানে হলো দুটো আলাদা team এবং দুটো আলাদা codebase রক্ষণাবেক্ষণ করতে হয় — যা অনেক ব্যয়বহুল এবং সময়সাপেক্ষ।

React Native এই সমস্যার সমাধান দেয়। Facebook (এখন Meta) ২০১৫ সালে React Native তৈরি করে। এটি দিয়ে JavaScript এবং React ব্যবহার করে একটিমাত্র codebase থেকে Android এবং iOS — উভয় প্ল্যাটফর্মের জন্য অ্যাপ তৈরি করা যায়।

React Native কিভাবে কাজ করে?

React Native একটি bridge ব্যবহার করে JavaScript কোড এবং Native (platform-specific) কোডের মধ্যে যোগাযোগ স্থাপন করে। আপনি যখন React Native-এ একটি <View> component ব্যবহার করেন, সেটি Android-এ একটি native View এবং iOS-এ একটি UIView হিসেবে render হয়।

সহজ ভাষায়: আপনি JavaScript লেখেন, React Native সেটাকে native components-এ রূপান্তরিত করে। ফলে অ্যাপটি সত্যিকারের native অ্যাপের মতো দ্রুত এবং smooth কাজ করে।

React Native-এর সুবিধা

- একটি codebase থেকে Android ও iOS উভয় প্ল্যাটফর্মে অ্যাপ তৈরি
- Web developer-রা সহজেই শিখতে পারেন (React-এর মতো syntax)
- Hot Reload — কোড পরিবর্তন করলে সাথে সাথে ফলাফল দেখা যায়
- বিশাল community এবং প্রচুর third-party library
- Facebook, Instagram, Airbnb, Microsoft-এর মতো বড় কোম্পানিগুলো ব্যবহার করে

React Native-এর সীমাবদ্ধতা

- Purely native app-এর তুলনায় performance কিছুটা কম হতে পারে
- Platform-specific কিছু feature implement করতে native কোড লিখতে হতে পারে
- Debugging কখনো কখনো জটিল হতে পারে

১.২ Expo কী?

Expo হলো React Native-এর উপরে তৈরি একটি framework এবং platform। Expo ব্যবহার করলে React Native অ্যাপ তৈরি করা অনেক সহজ হয়ে যায়।

ধরুন আপনি একটি নতুন বাড়ি বানাতে চান। React Native হলো সিমেন্ট, বালি, ইট — কাঁচামাল। Expo হলো একটি তৈরি বাড়ি নির্মাণ কিট যেখানে সব tools, materials প্যাকেজ করা আছে। আপনাকে আর আলাদা করে প্রতিটি জিনিস সংগ্রহ করতে হয় না।

Expo কী কী সুবিধা দেয়?

- Setup সহজ — Android Studio বা Xcode ইন্সটল না করেও ডেভেলপমেন্ট করা যায়
- Expo Go অ্যাপ দিয়ে সরাসরি ফোনে test করা যায়
- Camera, GPS, Push Notification সহ অনেক API built-in
- EAS (Expo Application Services) দিয়ে সহজে build ও deploy করা যায়
- Expo Router — file-based routing সিস্টেম (এই বইয়ের মূল বিষয়)

Expo-এর দুটি মোড

Expo দুটি মোডে কাজ করে:

1. Managed Workflow: Expo সব কিছু নিয়ন্ত্রণ করে। আপনাকে native কোডের চিন্তা করতে হয় না। Beginners-এর জন্য আদর্শ।
2. Bare Workflow: আপনার native code-এ সম্পূর্ণ access থাকে। Advanced users-এর জন্য।

এই বইয়ে আমরা Managed Workflow ব্যবহার করব।

> 💡 **জেনে রাখুন:** Expo SDK নিয়মিত আপডেট হয়। এই বইয়ে Expo SDK 51+ এবং Expo Router v3+ ব্যবহার করা হয়েছে।

১.৩ React Native vs Expo — পার্থক্য কী?

অনেকেই বিভ্রান্ত হন — React Native ব্যবহার করব নাকি Expo? নিচের তুলনা দেখুন:

বিষয়	React Native (bare)	Expo (Managed)
Setup জটিলতা	বেশি (Android Studio, Xcode লাগে)	কম (npx create-expo-app)
Native Module	সম্পূর্ণ access	সীমিত (কিন্তু যথেষ্ট)
App Size	ছোট	কিছুটা বড়
Beginner Friendly	না	হ্যাঁ
Build & Deploy	জটিল	EAS দিয়ে সহজ
File-based Routing	নেই (আলাদা library লাগে)	Expo Router built-in

১.৪ আপনার প্রথম React Native ধারণা

React Native-এ কাজ করার আগে কিছু মৌলিক বিষয় জানা দরকার:

Components

React Native-এ সব কিছুই Component দিয়ে তৈরি। Web-এ যেমন div, span, p ট্যাগ আছে, React Native-এ তেমনি আছে:

- View — div-এর মতো, container হিসেবে কাজ করে
- Text — যেকোনো text দেখানোর জন্য
- Image — ছবি দেখানোর জন্য
- TextInput — input field-এর জন্য
- TouchableOpacity / Pressable — button বানানোর জন্য
- ScrollView — scroll করার জন্য
- FlatList — বড় list render করার জন্য

StyleSheet

React Native-এ CSS নেই। পরিবর্তে StyleSheet API ব্যবহার করা হয়। Syntax অনেকটা CSS-এর মতোই, কিন্তু camelCase-এ লিখতে হয়।

```
// React Native StyleSheet উদাহরণ
import { View, Text, StyleSheet } from 'react-native';

export default function HelloScreen() {
  return (
    // View হলো container (div-এর মতো)
    <View style={styles.container}>
      <Text style={styles.title}>হ্যালো, Expo!</Text>
      <Text style={styles.subtitle}>আমার প্রথম React Native অ্যাপ</Text>
    </View>
  );
}

// StyleSheet দিয়ে style define করা হয়
const styles = StyleSheet.create({
  container: {
    flex: 1,              // পুরো স্ক্রিন জুড়ে
    alignItems: 'center',  // horizontal center
    justifyContent: 'center', // vertical center
    backgroundColor: '#f0f4f8',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#1a365d',
```

```
    marginBottom: 8,
  },
  subtitle: {
    fontSize: 16,
    color: '#718096',
  },
});
```

উপরের কোডটি বিশ্লেষণ করি: View কম্পোনেন্টটি পুরো স্ক্রিন জুড়ে বিস্তৃত (flex: 1)। alignItems এবং justifyContent দিয়ে content কে স্ক্রিনের মাঝখানে রাখা হয়েছে। Text কম্পোনেন্টে fontSize, fontWeight, color ইত্যাদি দিয়ে typography ঠিক করা হয়েছে।

১.৫ Flexbox — React Native-এর Layout System

React Native-এ layout করার জন্য Flexbox ব্যবহার হয়। Web Flexbox-এর সাথে কিছু পার্থক্য আছে:

- Default flexDirection হলো 'column' (web-এ 'row')
- flex: 1 মানে উপলব্ধ সব জায়গা নেওয়া
- alignItems এবং justifyContent দিয়ে alignment নিয়ন্ত্রণ

```
// Flexbox উদাহরণ
const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',     // items পাশাপাশি সাজাবে
    justifyContent: 'space-between', // সমান দূরত্বে
    alignItems: 'center',     // vertical center
    padding: 16,
  },
  column: {
    flexDirection: 'column', // default — items উপর-নিচে
    flex: 1,
  },
});
```

১.৬ Key Takeaways — অধ্যায় ১

💡 **মূল বিষয়সমূহ:** এই অধ্যায়ে আমরা যা শিখলাম:

- React Native হলো JavaScript দিয়ে cross-platform mobile app তৈরির framework
- Expo হলো React Native-এর উপরে তৈরি একটি সহজ platform
- Expo Managed Workflow beginners-এর জন্য সবচেয়ে উপযুক্ত
- React Native-এ View, Text, Image ইত্যাদি native components ব্যবহার হয়
- StyleSheet API দিয়ে CSS-এর মতো styling করা যায়
- Layout-এর জন্য Flexbox ব্যবহার করা হয়

১.৭ Common Mistakes

⚠ **সাধারণ ভুল ১:** React Native-এ HTML tag (, <p>,) ব্যবহার করা। সবসময় View, Text ইত্যাদি React Native components ব্যবহার করুন।

⚠ **সাধারণ ভুল ২:** Text-এর বাইরে সরাসরি string রাখা। React Native-এ সব text অবশ্যই <Text> component-এর ভেতরে থাকতে হবে।

⚠ **সাধারণ ভুল ৩:** StyleSheet-এ CSS-এর মতো hyphen ব্যবহার করা (যেমন: background-color)। React Native-এ camelCase লিখতে হয় (backgroundColor)।

১.৮ Best Practices

- সবসময় StyleSheet.create() ব্যবহার করুন — inline style এড়িয়ে চলুন
- Component গুলো ছোট ছোট রাখুন, প্রতিটি component একটিই কাজ করুক
- Flexbox ভালোভাবে শিখুন — React Native layout-এর ভিত্তি

১.৯ অনুশীলন (Exercise)

নিচের কাজগুলো করুন:

3. একটি নতুন Expo project তৈরি করুন (আমরা অধ্যায় ৩-এ শিখব, কিন্তু এখন থেকে চিন্তা শুরু করুন)
4. একটি simple স্ক্রিন তৈরি করুন যেখানে আপনার নাম, বিশ্ববিদ্যালয়ের নাম এবং বিভাগের নাম দেখাবে
5. Flexbox ব্যবহার করে তিনটি বক্স পাশাপাশি সাজান

অধ্যায় ২: Expo Router কী এবং কেন ব্যবহার করবেন

এই অধ্যায়ে আমরা জানব Navigation কী, Expo Router কিভাবে কাজ করে, এবং পুরনো React Navigation-এর চেয়ে এটি কেন বেশি সুবিধাজনক।

২.১ Navigation কী এবং কেন দরকার?

একটি মোবাইল অ্যাপে সাধারণত অনেকগুলো স্ক্রিন থাকে। যেমন: Home Screen, Profile Screen, Settings Screen, Product Detail Screen ইত্যাদি। এক স্ক্রিন থেকে অন্য স্ক্রিনে যাওয়াকেই Navigation বলে।

ওয়েব ব্রাউজারে আমরা URL পরিবর্তন করে এক পেজ থেকে অন্য পেজে যাই (যেমন: /home, /about, /products/123)। মোবাইল অ্যাপেও এরকম একটি system দরকার যা track রাখবে:

- ব্যবহারকারী এখন কোন স্ক্রিনে আছেন
- Back button চাপলে কোথায় ফিরে যাবেন
- কিভাবে data এক স্ক্রিন থেকে অন্য স্ক্রিনে পাঠানো যাবে

২.২ পুরনো পদ্ধতি — React Navigation

Expo Router আসার আগে React Navigation ছিল সবচেয়ে জনপ্রিয় navigation library। React Navigation এখনো ব্যাপকভাবে ব্যবহৃত হয়, তবে এর কিছু সমস্যা আছে।

React Navigation-এ একটি simple setup:

```
// React Navigation — পুরনো পদ্ধতি
// app.js ফাইলে সব routes manually define করতে হয়
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import ProfileScreen from './screens/ProfileScreen';
import ProductScreen from './screens/ProductScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    // প্রতিটি screen আলাদাভাবে register করতে হয়
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Profile" component={ProfileScreen} />
        <Stack.Screen name="Product" component={ProductScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
```

```
}
```

এই পদ্ধতিতে সমস্যা হলো প্রতিটি নতুন screen যোগ করতে app.js ফাইলে এসে route register করতে হয়। অ্যাপ বড় হলে এই ফাইলটি অনেক লম্বা হয়ে যায়।

২.৩ Expo Router — আধুনিক পদ্ধতি

Expo Router একটি file-based routing system। এর মানে হলো আপনার file structure-ই আপনার routes নির্ধারণ করে — ঠিক যেমন Next.js web framework-এ হয়।

Expo Router-এ একই কাজ কত সহজে হয়:

```
// Expo Router — আধুনিক পদ্ধতি
// কোনো manual route registration নেই!
// শুধু সঠিক জায়গায় ফাইল রাখুন

app/
├── index.js          →  Route: /           (Home)
├── profile.js        →  Route: /profile    (Profile)
└── products/
    └── [id].js       →  Route: /products/123   (Dynamic)
```

এখানে দেখুন — কোনো configuration নেই, কোনো manual registration নেই। শুধু সঠিক নামে ফাইল তৈরি করুন, Expo Router বাকি সব নিজেই করে নেবে।

২.৪ Expo Router কিভাবে কাজ করে?

Expo Router-এর কাজ করার পদ্ধতি বোঝার জন্য একটি বাস্তব উদাহরণ দেখি।

File Structure থেকে Routes:

```
myapp/
├── app/
│   ├── _layout.js       →  Root layout (সব screen-এ common)
│   ├── index.js         →  Route: /
│   ├── about.js         →  Route: /about
│   ├── (tabs)/
│   │   ├── _layout.js   →  Tab layout
│   │   ├── home.js      →  Tab: Home
│   │   └── profile.js   →  Tab: Profile
│   └── products/
│       ├── index.js     →  Route: /products
│       └── [id].js      →  Route: /products/:id
```

Special Files:

- _layout.js — ঐ folder-এর সব screen-এ যে layout common থাকবে
- index.js — folder-এর default route (যেমন /products folder-এ /products route)
- [id].js — dynamic route, যেকোনো value নিতে পারে
- (tabs)/ — parentheses মানে এটি URL-এ দেখাবে না, শুধু grouping-এর জন্য

২.৫ React Navigation vs Expo Router — বিস্তারিত তুলনা

বৈশিষ্ট্য	React Navigation	Expo Router
Route Definition	Code-এ manually	File structure স্বয়ংক্রিয়
Deep Linking	জটিল setup	Built-in, স্বয়ংক্রিয়
URL Sharing	সীমিত	সম্পূর্ণ URL support
Web Support	আলাদা setup	Universal (Android/iOS/Web)
TypeScript	Manual types	Auto-generated
Learning Curve	মাঝারি	সহজ (Next.js-এর মতো)
SEO (Web)	নেই	আছে
Shared Routes	জটিল	সহজ

২.৬ Expo Router-এর Universal Routing

Expo Router-এর একটি বিশেষ সুবিধা হলো Universal Routing। একই codebase থেকে Android, iOS এবং Web — তিনটি প্ল্যাটফর্মেই route কাজ করে।

ধরুন আপনার অ্যাপে একটি product detail page আছে, route হলো /products/123। এই route:

- Android অ্যাপে — screen navigation হিসেবে কাজ করে
- iOS অ্যাপে — native navigation হিসেবে কাজ করে
- Web browser-এ — https://yourapp.com/products/123 URL হিসেবে কাজ করে

> 💡 **Universal App:** Expo Router দিয়ে একটি অ্যাপ তৈরি করলে সেটি একই সাথে Android, iOS, এবং Web-এ কাজ করে। এটি React Native-এর 'write once, run anywhere' দর্শনকে পূর্ণমাত্রায় বাস্তবায়ন করে।

২.৭ Key Takeaways — অধ্যায় ২

- Navigation হলো মোবাইল অ্যাপে এক স্ক্রিন থেকে অন্য স্ক্রিনে যাওয়ার system
- Expo Router একটি file-based routing system — Next.js-এর মতো
- File structure-ই routes নির্ধারণ করে — কোনো manual registration নেই

- React Navigation-এর চেয়ে Expo Router সহজ এবং আধুনিক
- Expo Router Android, iOS, এবং Web — তিনটি প্ল্যাটফর্মে কাজ করে

২.৮ Common Mistakes

> ⚠ **ভুল ধারণা:** অনেকে মনে করেন Expo Router = React Navigation। আসলে এগুলো আলাদা। Expo Router file-based, React Navigation code-based। তবে Expo Router ভেতরে React Navigation ব্যবহার করে।
>
> ⚠ **ফাইল নামকরণ:** Expo Router-এ file name-ই route হয়। তাই ফাইলের নাম দেওয়ার সময় সতর্ক থাকুন। ছোট হাতে এবং hyphen ব্যবহার করাই convention।

২.৯ অনুশীলন

চিন্তা করুন: আপনি একটি ই-কমার্স অ্যাপ বানাচ্ছেন। কোন কোন screen দরকার হবে? সেগুলোর file structure কেমন হবে Expo Router-এ? কাগজে লিখে দেখান।

অধ্যায় ৩: Installation এবং Setup

এই অধ্যায়ে আমরা হাতে-কলমে Expo Router দিয়ে একটি প্রজেক্ট তৈরি করব। আপনার computer-এ Node.js থাকলেই শুরু করতে পারবেন।

৩.১ প্রয়োজনীয় সরঞ্জাম (Prerequisites)

ইন্সটল করুন:

6. Node.js (v18 বা তার উপরে) — https://nodejs.org থেকে download করুন
7. VS Code — https://code.visualstudio.com (recommended editor)
8. Expo Go App — আপনার Android বা iOS ফোনে Google Play / App Store থেকে
9. Git — https://git-scm.com

Version যাচাই করুন:

```
# Terminal / Command Prompt খুলুন
node --version    # v18.0.0 বা তার উপরে হওয়া উচিত
npm --version     # 9.0.0 বা তার উপরে
npx --version     # npm-এর সাথে আসে
```

৩.২ নতুন Expo Project তৈরি

```
# Terminal-এ এই command চালান:
npx create-expo-app@latest MyFirstApp --template blank

# project folder-এ প্রবেশ করুন:
cd MyFirstApp

# dependency install করুন:
npm install
```

এখানে MyFirstApp হলো আপনার project-এর নাম। আপনি যেকোনো নাম দিতে পারেন। --template blank মানে একটি minimal template নেওয়া হচ্ছে।

৩.৩ Expo Router Install করা

নতুন Expo project-এ Expo Router সাধারণত include থাকে। কিন্তু যদি না থাকে বা manually install করতে চান:

```
# Expo Router এবং প্রয়োজনীয় dependencies install
```

```
npx expo install expo-router react-native-safe-area-context react-native-
screens expo-linking expo-constants expo-status-bar

# অথবা সরাসরি Expo Router template দিয়ে project তৈরি:
npx create-expo-app@latest MyRouterApp --template expo-template-blank-
typescript
```

৩.৪ package.json Configuration

package.json ফাইলটি খুলুন এবং main entry point পরিবর্তন করুন:

```
{
  "name": "myfirstapp",
  "version": "1.0.0",
  "main": "expo-router/entry",  // এটি অবশ্যই থাকতে হবে
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web"
  },
  "dependencies": {
    "expo": "~51.0.0",
    "expo-router": "~3.5.0",
    "react": "18.2.0",
    "react-native": "0.74.0",
    "react-native-safe-area-context": "4.10.1",
    "react-native-screens": "3.31.1"
  }
}
```

৩.৫ app.json Configuration

app.json ফাইলে scheme যোগ করতে হবে। এটি Deep Linking-এর জন্য দরকার:

```
{
  "expo": {
    "name": "MyFirstApp",
    "slug": "myfirstapp",
    "version": "1.0.0",
    "scheme": "myapp",        // Deep linking-এর জন্য জরুরি
    "web": {
      "bundler": "metro",
      "output": "static",
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
      "expo-router"            // Expo Router plugin যোগ করুন
    ],
    "experiments": {
      "typedRoutes": true      // TypeScript route types (optional)
```

```
    }
  }
}
```

৩.৬ প্রথম File Structure তৈরি

এখন app/ folder তৈরি করুন এবং প্রথম কিছু ফাইল যোগ করুন:

```
MyFirstApp/
├── app/                    # এই folder-ই সব routes
│   ├── _layout.js          # Root layout
│   ├── index.js            # Home screen (/)
│   └── about.js            # About screen (/about)
├── assets/                 # Images, fonts ইত্যাদি
├── components/             # Reusable components
├── app.json
└── package.json
```

app/_layout.js তৈরি করুন:

```
// app/_layout.js
// এটি Root Layout — সব screen-এ এটি wrap করে থাকবে
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    // Stack Navigator — এক screen-এর উপর আরেক screen stack হয়
    <Stack>
      <Stack.Screen
        name="index"
        options={{ title: 'হোম' }}   // Screen-এর title
      />
      <Stack.Screen
        name="about"
        options={{ title: 'আমাদের সম্পর্কে' }}
      />
    </Stack>
  );
}
```

app/index.js তৈরি করুন:

```
// app/index.js
// এটি Home Screen — Route: /
import { View, Text, StyleSheet, Pressable } from 'react-native';
import { Link } from 'expo-router';  // Expo Router-এর Link component

export default function HomeScreen() {
  return (
```

```
    <View style={styles.container}>
      <Text style={styles.title}>🏠 হোম স্ক্রিন</Text>
      <Text style={styles.subtitle}>Expo Router-এ স্বাগতম!</Text>

      {/* Link দিয়ে অন্য screen-এ navigate করুন */}
      <Link href="/about" style={styles.link}>
        আমাদের সম্পর্কে জানুন →
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: '#f0f4f8',
    padding: 20,
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    color: '#1a365d',
    marginBottom: 8,
  },
  subtitle: {
    fontSize: 16,
    color: '#718096',
    marginBottom: 32,
    textAlign: 'center',
  },
  link: {
    backgroundColor: '#3182ce',
    color: '#ffffff',
    padding: 16,
    borderRadius: 8,
    fontSize: 16,
    fontWeight: '600',
  },
});
```

app/about.js তৈরি করুন:

```
// app/about.js
// এটি About Screen — Route: /about
import { View, Text, StyleSheet } from 'react-native';
import { Link } from 'expo-router';

export default function AboutScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>ℹ️ আমাদের সম্পর্কে</Text>
```

```
      <Text style={styles.body}>
        এই অ্যাপটি Expo Router শেখার জন্য তৈরি করা হয়েছে।
      </Text>

      {/* হোমে ফিরে যাওয়ার link */}
      <Link href="/" style={styles.link}>
        ← হোমে ফিরুন
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: '#f0f4f8',
    padding: 20,
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    color: '#1a365d',
    marginBottom: 16,
  },
  body: {
    fontSize: 16,
    color: '#4a5568',
    textAlign: 'center',
    marginBottom: 32,
    lineHeight: 24,
  },
  link: {
    color: '#3182ce',
    fontSize: 16,
    fontWeight: '600',
  },
});
```

৩.৭ অ্যাপ চালু করা

```
# Terminal-এ project folder থেকে:
npx expo start

# নিচের options দেখবেন:
#  › Press a │ open Android
#  › Press i │ open iOS simulator
#  › Press w │ open web
#  › Scan QR code with Expo Go (Android) or Camera app (iOS)
```

আপনার ফোনে Expo Go অ্যাপ খুলুন এবং QR code scan করুন। অ্যাপটি আপনার ফোনে চলবে! Home screen দেখবেন, 'আমাদের সম্পর্কে জানুন' button-এ চাপলে About screen-এ যাবেন।

৩.৮ VS Code Extensions

ডেভেলপমেন্ট সহজ করতে এই VS Code extensions install করুন:

- ES7+ React/Redux/React-Native snippets — দ্রুত code লিখতে
- Prettier — Code formatter
- ESLint — Code quality checker
- React Native Tools — debugging সহজ করে
- Auto Rename Tag — JSX tag rename করতে

৩.৯ Key Takeaways — অধ্যায় ৩

- npx create-expo-app দিয়ে সহজেই নতুন project তৈরি হয়
- package.json-এ main: 'expo-router/entry' সেট করতে হয়
- app.json-এ scheme এবং expo-router plugin যোগ করতে হয়
- app/ folder-এর files স্বয়ংক্রিয়ভাবে routes হয়ে যায়
- npx expo start দিয়ে development server চালু হয়

৩.১০ Common Mistakes

> ⚠ **main entry point:** package.json-এ main: 'expo-router/entry' না দিলে Expo Router কাজ করবে না। এটি সবচেয়ে সাধারণ ভুল।
>
> ⚠ **app/ folder:** সব route files অবশ্যই app/ folder-এর ভেতরে থাকতে হবে। বাইরে রাখলে route হিসেবে কাজ করবে না।
>
> ⚠ **scheme missing:** app.json-এ scheme না দিলে deep linking এবং web support কাজ করবে না।

৩.১১ অনুশীলন

10. উপরের steps follow করে আপনার computer-এ একটি Expo Router project তৈরি করুন
11. Home এবং About screen তৈরি করুন এবং ফোনে বা emulator-এ চালিয়ে দেখুন
12. আরেকটি screen যোগ করুন — Contact screen, route হবে /contact

অধ্যায় ৪: File-based Routing ব্যাখ্যা

এই অধ্যায়ে Expo Router-এর সবচেয়ে গুরুত্বপূর্ণ concept — File-based Routing — বিস্তারিতভাবে শিখব। এটি বুঝলে পুরো Expo Router সহজ হয়ে যাবে।

৪.১ File-based Routing কী?

Traditional React Native-এ routes manually define করতে হয়। কিন্তু Expo Router-এ ফাইলের নাম এবং অবস্থানই route নির্ধারণ করে।

একটি সহজ উদাহরণ দিয়ে বুঝি: ধরুন আপনি একটি Library-এর catalog বানাচ্ছেন। Traditional পদ্ধতিতে আপনাকে একটি লম্বা list বানাতে হতো: 'শেলফ A-তে রবীন্দ্রনাথ, শেলফ B-তে নজরুল...' কিন্তু file-based পদ্ধতিতে আপনি শুধু সঠিক শেলফে বই রাখলেই হয় — catalog স্বয়ংক্রিয়ভাবে তৈরি হয়।

৪.২ Basic Routing Rules

নিয়ম ১: File name = Route name

```
app/
├── index.js       → Route: /
├── about.js       → Route: /about
├── contact.js     → Route: /contact
└── blog.js        → Route: /blog
```

নিয়ম ২: Folder = Nested Route

```
app/
├── index.js            → Route: /
└── products/
    ├── index.js        → Route: /products
    ├── featured.js     → Route: /products/featured
    └── sale.js         → Route: /products/sale
```

নিয়ম ৩: index.js মানে folder-এর default route

products/index.js মানে route /products — index.js ফাইলটি folder-এর নাম হিসেবে দেখায়।

৪.৩ Special File Naming Conventions

Dynamic Routes — [param].js

```
app/
└── users/
    ├── index.js         → Route: /users (সব user দেখাবে)
```

```
    └── [id].js          → Route: /users/123, /users/456, যেকোনো id
```

```
// app/users/[id].js
import { useLocalSearchParams } from 'expo-router';
import { View, Text } from 'react-native';

export default function UserProfile() {
  // URL-এর id parameter এখানে পাওয়া যাবে
  const { id } = useLocalSearchParams();

  return (
    <View>
      <Text>ব্যবহারকারী ID: {id}</Text>
    </View>
  );
}
```

Layout Files — _layout.js

_layout.js ফাইলটি ঐ folder-এর সব screen-এ wrapper হিসেবে কাজ করে। এটি route হিসেবে কাজ করে না।

```
app/
├── _layout.js          # সব screen-এর root layout
├── index.js
└── settings/
    ├── _layout.js      # settings folder-এর layout
    ├── profile.js
    └── security.js
```

Route Groups — (group)

Parentheses দিয়ে folder তৈরি করলে সেটি URL-এ দেখায় না। শুধু organization-এর জন্য ব্যবহার হয়:

```
app/
├── (auth)/              # URL-এ দেখাবে না
│   ├── login.js         → Route: /login (NOT /auth/login)
│   └── register.js      → Route: /register
└── (main)/             # URL-এ দেখাবে না
    ├── home.js          → Route: /home
    └── profile.js       → Route: /profile
```

> 💡 **Route Groups:** Route groups দিয়ে আপনি আলাদা layout apply করতে পারেন বিভিন্ন section-এ। যেমন auth pages-এ কোনো header থাকবে না, কিন্তু main pages-এ থাকবে।

Catch-all Routes — [...param].js

যেকোনো depth-এর route match করতে:

```
app/
└── docs/
    └── [...slug].js    → Match: /docs/intro
                        → Match: /docs/api/users
                        → Match: /docs/guide/advanced/routing
```

```
// app/docs/[...slug].js
import { useLocalSearchParams } from 'expo-router';

export default function DocsPage() {
  const { slug } = useLocalSearchParams();
  // slug হবে একটি array: ['api', 'users']
  const path = Array.isArray(slug) ? slug.join('/') : slug;

  return <Text>Documentation: {path}</Text>;
}
```

৪.৪ একটি বাস্তব Project Structure

একটি e-commerce অ্যাপের complete file structure দেখুন:

```
ShopApp/
└── app/
    ├── _layout.js              # Root layout (global header, etc.)
    │
    ├── (auth)/                 # Authentication group
    │   ├── _layout.js          # Auth-specific layout (no header)
    │   ├── login.js            # /login
    │   └── register.js         # /register
    │
    ├── (tabs)/                 # Main tab navigation group
    │   ├── _layout.js          # Tab bar layout
    │   ├── index.js            # / (Home tab)
    │   ├── search.js           # /search (Search tab)
    │   ├── cart.js             # /cart (Cart tab)
    │   └── profile.js          # /profile (Profile tab)
    │
    ├── products/
    │   ├── index.js            # /products (All products)
    │   ├── [id].js             # /products/123 (Product detail)
    │   └── category/
    │       └── [slug].js       # /products/category/electronics
    │
    ├── orders/
    │   ├── index.js            # /orders (Order history)
    │   └── [orderId].js        # /orders/ORD-001
    │
    └── +not-found.js           # 404 page
```

৪.৫ +not-found.js — 404 Page

Expo Router-এ +not-found.js দিয়ে custom 404 page তৈরি করা যায়:

```
// app/+not-found.js
import { View, Text, StyleSheet } from 'react-native';
import { Link } from 'expo-router';

export default function NotFoundScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.emoji}>☹</Text>
      <Text style={styles.title}>পেজ পাওয়া যায়নি!</Text>
      <Text style={styles.subtitle}>
        আপনি যে পেজটি খুঁজছেন সেটি নেই।
      </Text>
      <Link href="/" style={styles.homeLink}>
        হোমে ফিরে যান
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center',
padding: 20 },
  emoji: { fontSize: 64, marginBottom: 16 },
  title: { fontSize: 24, fontWeight: 'bold', color: '#1a365d', marginBottom:
8 },
  subtitle: { fontSize: 16, color: '#718096', textAlign: 'center',
marginBottom: 24 },
  homeLink: { color: '#3182ce', fontSize: 16, fontWeight: '600' },
});
```

৪.৬ Route Naming Best Practices

- ছোট হাতে লিখুন (lowercase): about.js, not About.js
- hyphen ব্যবহার করুন (kebab-case): user-profile.js, not userProfile.js
- অর্থপূর্ণ নাম দিন: product-detail.js, not screen3.js
- Folder structure যুক্তিসংগত রাখুন: সম্পর্কিত routes একই folder-এ

৪.৭ Key Takeaways — অধ্যায় ৪

- File name = Route URL — যে নামে ফাইল, সেই নামে route
- index.js মানে folder-এর default route
- [param].js মানে dynamic route — যেকোনো value accept করে

- _layout.js URL-এ দেখায় না, wrapper হিসেবে কাজ করে
- (group) folder URL-এ দেখায় না, organization-এর জন্য
- +not-found.js দিয়ে custom 404 page বানানো যায়

৪.৮ Common Mistakes

⚠ **Case sensitivity:** File name case-sensitive। about.js এবং About.js আলাদা route। সবসময় lowercase ব্যবহার করুন।

⚠ **index.js vs folder:** products/ folder-এ index.js থাকলে /products route হয়। কিন্তু products.js ফাইল থাকলেও /products route হয়। দুটো একসাথে রাখবেন না।

৪.৯ অনুশীলন

13. একটি Blog অ্যাপের file structure design করুন (posts list, post detail, categories, about page সহ)
14. Dynamic route তৈরি করুন যেখানে post ID দিয়ে specific post দেখা যাবে
15. Auth group এবং Main group আলাদা করুন

অধ্যায় ৫: Navigation Basics — Link এবং useRouter

এখন আমরা শিখব কিভাবে এক screen থেকে অন্য screen-এ navigate করা যায়। Expo Router-এ এটি করার দুটি প্রধান উপায় আছে: Link component এবং useRouter hook।

৫.১ Link Component

Link হলো Expo Router-এর সবচেয়ে সহজ navigation method। Web-এর <a> tag-এর মতো কাজ করে।

Basic Link ব্যবহার:

```
import { Link } from 'expo-router';
import { View, Text } from 'react-native';

export default function HomeScreen() {
  return (
    <View>
      <Text>হোম স্ক্রিন</Text>

      {/* Simple navigation */}
      <Link href="/about">আমাদের সম্পর্কে</Link>

      {/* Dynamic route-এ navigate */}
      <Link href="/products/123">Product 123 দেখুন</Link>

      {/* External link */}
      <Link href="https://expo.dev">Expo ওয়েবসাইট</Link>
    </View>
  );
}
```

Link-এর asChild prop

Link by default একটি Text element render করে। কিন্তু asChild prop ব্যবহার করলে আপনার নিজের component use করতে পারবেন:

```
import { Link } from 'expo-router';
import { Pressable, Text, StyleSheet } from 'react-native';

export default function HomeScreen() {
  return (
    <View>
      {/* asChild দিয়ে custom button */}
      <Link href="/profile" asChild>
        <Pressable style={styles.button}>
          <Text style={styles.buttonText}>প্রোফাইল দেখুন</Text>
```

```
        </Pressable>
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: '#3182ce',
    padding: 16,
    borderRadius: 8,
    alignItems: 'center',
  },
  buttonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: '600',
  },
});
```

Link-এর replace prop

replace prop দিলে navigation history-তে নতুন entry যোগ না হয়ে replace হয়। Login screen-এর পরে Home screen-এ যাওয়ার সময় এটি কাজে লাগে:

```
// Login successful হলে Home-এ যান, কিন্তু back চাপলে Login-এ না ফিরে যেন
<Link href="/(tabs)/home" replace>
  লগইন করুন
</Link>
```

৫.২ useRouter Hook

useRouter hook ব্যবহার করলে JavaScript code থেকে programmatically navigate করা যায়। যেমন: button press করার পর কিছু কাজ করে তারপর navigate করা।

useRouter-এর methods:

```
import { useRouter } from 'expo-router';
import { View, Text, Pressable } from 'react-native';

export default function LoginScreen() {
  const router = useRouter();

  const handleLogin = async () => {
    // প্রথমে login logic চালান
    const success = await loginUser();

    if (success) {
      // navigate() — navigation stack-এ push করে
      router.push('/home');
```

```
      // replace() — current screen replace করে (back যাওয়া যাবে না)
      // router.replace('/home');

      // back() — আগের screen-এ ফিরে যায়
      // router.back();
    }
  };

  return (
    <View>
      <Pressable onPress={handleLogin}>
        <Text>লগইন করুন</Text>
      </Pressable>
    </View>
  );
}
```

useRouter-এর সব methods:

- router.push(href) — নতুন screen push করে, back button কাজ করে
- router.replace(href) — current screen replace করে, back যাওয়া যায় না
- router.back() — আগের screen-এ ফিরে যায়
- router.canGoBack() — back যাওয়া সম্ভব কিনা check করে
- router.dismiss() — modal বা sheet বন্ধ করে
- router.navigate(href) — smart navigation (route already active থাকলে push করে না)

৫.৩ Query Parameters পাঠানো

Navigate করার সময় extra data পাঠানো যায় query parameters হিসেবে:

```
// Navigate করার সময় params পাঠান
router.push({
  pathname: '/products/[id]',
  params: { id: '123', category: 'electronics' }
});

// অথবা Link-এ:
<Link href={{ pathname: '/products/[id]', params: { id: '123' } }}>
  Product দেখুন
</Link>
```

Params receive করা — useLocalSearchParams:

```
// app/products/[id].js
import { useLocalSearchParams } from 'expo-router';
```

```
import { View, Text } from 'react-native';

export default function ProductDetail() {
  // URL params এবং query params সব এখানে পাওয়া যাবে
  const { id, category } = useLocalSearchParams();

  return (
    <View>
      <Text>Product ID: {id}</Text>
      <Text>Category: {category}</Text>
    </View>
  );
}
```

৫.৪ usePathname এবং useSegments

Current route সম্পর্কে জানতে এই hooks ব্যবহার করুন:

```
import { usePathname, useSegments } from 'expo-router';

export default function SomeComponent() {
  // বর্তমান path পান
  const pathname = usePathname();
  // উদাহরণ: '/products/123'

  // path segments পান (array হিসেবে)
  const segments = useSegments();
  // উদাহরণ: ['products', '123']

  console.log('Current path:', pathname);
  console.log('Segments:', segments);

  return (
    <View>
      <Text>আপনি আছেন: {pathname}</Text>
    </View>
  );
}
```

৫.৫ একটি Complete Navigation উদাহরণ

এখন একটি বাস্তব উদাহরণ দেখি — একটি simple product browsing app:

app/(tabs)/_layout.js — Tab Navigation:

```
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

export default function TabsLayout() {
```

```
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: '#3182ce',
        tabBarInactiveTintColor: '#718096',
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: 'হোম',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="home" size={size} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="products"
        options={{
          title: 'পণ্য',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="grid" size={size} color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="cart"
        options={{
          title: 'কার্ট',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="cart" size={size} color={color} />
          ),
        }}
      />
    </Tabs>
  );
}
```

app/(tabs)/products.js — Products List:

```
import { View, Text, FlatList, Pressable, StyleSheet } from 'react-native';
import { useRouter } from 'expo-router';

// Sample data
const products = [
  { id: '1', name: 'ল্যাপটপ', price: 45000 },
  { id: '2', name: 'স্মার্টফোন', price: 22000 },
  { id: '3', name: 'হেডফোন', price: 3500 },
  { id: '4', name: 'ট্যাবলেট', price: 18000 },
];

export default function ProductsScreen() {
  const router = useRouter();
```

```
  const handleProductPress = (productId) => {
    // Programmatic navigation with params
    router.push({
      pathname: '/product-detail/[id]',
      params: { id: productId }
    });
  };

  return (
    <View style={styles.container}>
      <Text style={styles.heading}>সব পণ্য</Text>
      <FlatList
        data={products}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable
            style={styles.productCard}
            onPress={() => handleProductPress(item.id)}
          >
            <Text style={styles.productName}>{item.name}</Text>
            <Text style={styles.productPrice}>৳ {item.price}</Text>
          </Pressable>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f7fafc', padding: 16 },
  heading: { fontSize: 22, fontWeight: 'bold', color: '#1a365d',
marginBottom: 16 },
  productCard: {
    backgroundColor: '#ffffff',
    padding: 16,
    borderRadius: 8,
    marginBottom: 8,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
    shadowRadius: 3,
    elevation: 2,
  },
  productName: { fontSize: 16, color: '#2d3748', fontWeight: '500' },
  productPrice: { fontSize: 15, color: '#3182ce', fontWeight: '600' },
});
```

৫.৬ Expo Router vs React Navigation Navigation পদ্ধতি

React Navigation-এ navigation করতে হতো props দিয়ে:

```
// React Navigation — পুরনো পদ্ধতি
// navigation prop automatically pass হতো
export default function HomeScreen({ navigation }) {
  return (
    <Button
      title="Profile"
      onPress={() => navigation.navigate('Profile', { userId: '123' })}
    />
  );
}

// Expo Router — আধুনিক পদ্ধতি
import { useRouter } from 'expo-router';
export default function HomeScreen() {
  const router = useRouter();
  return (
    <Button
      title="Profile"
      onPress={() => router.push({ pathname: '/profile', params: { userId:
'123' } })}
    />
  );
}
```

💡 **সুবিধা:** Expo Router-এ navigation prop pass করতে হয় না। যেকোনো component থেকে useRouter() ব্যবহার করলেই হয়।

৫.৭ Key Takeaways — অধ্যায় ৫

- Link component — declarative navigation (HTML anchor-এর মতো)
- useRouter hook — programmatic navigation (JS code থেকে)
- router.push() — নতুন screen, back button কাজ করে
- router.replace() — current screen replace, back যাওয়া যায় না
- useLocalSearchParams() — URL params এবং query params পাওয়া যায়
- usePathname(), useSegments() — current route info পাওয়া যায়

৫.৮ Common Mistakes

⚠ **Link-এর মধ্যে Text:** Link component নিজেই Text render করে। Link-এর মধ্যে আলাদা Text component না দিয়ে asChild ব্যবহার করুন যদি custom styling চান।

⚠ **push vs replace:** Login-এর পরে home-এ যাওয়ার সময় replace() ব্যবহার করুন, push() নয়। তাহলে user home থেকে back চাপলে login-এ ফিরে যাবে না।

> ⚠ **params type:** useLocalSearchParams() সবসময় string return করে। Number দরকার হলে parseInt() বা parseFloat() ব্যবহার করুন।

৫.৯ Best Practices

- Simple navigation-এ Link use করুন, complex logic-এ useRouter
- Sensitive data (password, token) params-এ পাঠাবেন না
- Replace() ব্যবহার করুন যেখানে back navigation desired নয়
- Deep linking-এর কথা মাথায় রেখে route structure design করুন

৫.১০ অনুশীলন

16. একটি screen তৈরি করুন যেখানে তিনটি product card আছে। প্রতিটি card-এ press করলে product detail screen-এ যাবে product ID পাঠিয়ে।
17. Detail screen-এ সেই ID দেখান এবং একটি 'কার্টে যোগ করুন' button রাখুন যেটি press করলে cart screen-এ navigate করবে।
18. usePathname() ব্যবহার করে current route দেখান header-এ।

পরবর্তী অধ্যায়সমূহ

অধ্যায় ১ থেকে ৫ সম্পন্ন হয়েছে। এই পাঁচটি অধ্যায়ে আমরা শিখেছি:

- React Native ও Expo-এর মূল ধারণা
- Expo Router কেন ব্যবহার করব এবং এটি কিভাবে কাজ করে
- Project setup এবং configuration
- File-based routing-এর সম্পূর্ণ নিয়মকানুন
- Link এবং useRouter দিয়ে navigation

পরবর্তী অধ্যায়গুলোতে আমরা শিখব: Dynamic Routing, Layouts, Stack/Tab/Drawer Navigation, Authentication, API Integration এবং আরো অনেক কিছু।

> 💡 **চালিয়ে যান:** প্রতিটি অধ্যায়ের Exercise অবশ্যই করুন। হাতে-কলমে practice ছাড়া শুধু পড়লে শেখা টিকে না!